

Ejercicios - Taller 1

1-) Tienes una lista de transacciones bancarias. Cada transacción es un objeto con la siguiente estructura:

```
1  enum TransactionType {
2      DEPOSITO,
3      RETIRO
4  }
5
6  class Transaction{
7      id: number;
8      type: TransactionType;
9      amount: number
10     date: string
11 }
```

La lista de transacciones es la siguiente

```
const transactions : Transaction[] = [
  { id: 1, type: TransactionType.DEPOSITO, amount: 250.00, date: "2024-05-01" },
  { id: 2, type : TransactionType.RETIRO, amount: 75.50, date: "2024-05-02"},
  { id: 3, type : TransactionType.DEPOSITO, amount: 150.00, date: "2024-05-03"},
  { id: 4, type : TransactionType.RETIRO, amount: 120.00, date: "2024-05-04"},
  { id: 5, type : TransactionType.RETIRO, amount: 50.25, date: "2024-05-05"},
  { id: 6, type : TransactionType.DEPOSITO, amount: 300.00, date: "2024-05-06"},
  { id: 7, type : TransactionType.RETIRO, amount: 200.00, date: "2024-05-07"},
  { id: 8, type : TransactionType.RETIRO, amount: 99.99, date: "2024-05-08"},
]
```

Tomando en cuenta esto, se necesita

- Para las transacciones de depósito se necesita aplicar un cargo adicional del 1.5% del monto de la transacción. Para esto, a cada transacción se le debe agregar el atributo "additional" con el monto del cargo a aplicar y también sumarlo al monto original. Después de esto deben obtener una suma de los montos de todas las transacciones que su monto sea mayor o igual que 100 después de aplicado el cargo.
- Para las transacciones de retiro se debe descontar un 0.5% del monto original que debe verse sobre el atributo amount y agregar el atributo "discount" con el monto descontado. Después obtener un promedio de los montos de las transacciones que hayan generado, por lo menos, un descuento equivalente a 0.5.
- Obtener un objeto con 2 atributos, retire y deposit. retire debe contener un arreglo con los retiros que se hayan realizado desde el 5 de mayo de 2024; deposit debe contener un arreglo con los depósitos realizados hasta el 4 de mayo de 2025. Esos arreglos deben contener objetos que tengan solamente el monto y la fecha de la transacción

Estos requerimientos deben ser mostrados por consola. Para esto no se puede utilizar estructuras iterativas tradicionales (while, do while y for), solo funciones de orden superior (map, filter, reduce).

2-) Tomando como referencia el arreglo de equipos de fútbol que se utilizó en la clase de funciones de orden superior, se requiere lo siguiente:

- Un objeto cuyo atributos sea el nombre de cada uno de los países a lo que pertenezcan los equipos, esos atributos deben contener un string con el nombre del equipo de ese país que haya ganado más recientemente una UCL.
- Un objeto cuyo atributos sea el nombre de cada uno de los países a lo que pertenezcan los equipos, esos atributos deben contener un promedio del atributo `cryLevel` de los equipos de ese país
- Un arreglo con los nombres de los equipos que tengan más tiempo sin ganar la UCL. Para determinar esto, saquen el promedio de años sin ganar, y los equipos que tengan más años sin ganar que el promedio, entran al arreglo.
- Sin incluir a España, indicar el país con más Champions conseguidas por sus equipos.

3. Se tiene un sistema de monitoreo de los servidores que tiene desplegados una empresa. Ese sistema de monitoreo te proporciona el estatus de cada uno de los servidores siguiendo la siguiente estructura

```
enum Region {
  US_EAST,
  US_WEST,
  EU_WEST,
  EU_NORDIC_EAST,
  LAN,
  LAS,
  BR
}

enum Status {
  UP,
  DOWN,
  DEGRADED
}

interface cloudMetrics {
  serviceName: string;
  status: Status;
  responseTime?: number;
  region: Region;
  date: string;
}
```

Al consultarlo el 02/05/2026, el resultado que te da es el siguiente:

```
const metrics : cloudMetrics[] = [
  { serviceName: "Queue Service", status: Status.DOWN, region: Region.US_EAST, date: "2026-03-06" },
  { serviceName: "NGINX", status: Status.UP, responseTime: 600, region: Region.US_EAST, date: "2026-04-06" },
  { serviceName: "Message Server", status: Status.DEGRADED, responseTime: 500, region: Region.US_WEST, date: "2026-02-28" },
  { serviceName: "Email server", status: Status.DOWN, region: Region.US_EAST, date: "2026-03-06" },
  { serviceName: "Kafka", status: Status.DOWN, region: Region.US_EAST, date: "2026-04-26" },
  { serviceName: "BE Service", status: Status.UP, responseTime: 120, region: Region.US_EAST, date: "2026-01-29" },
  { serviceName: "FE Service", status: Status.UP, responseTime: 90, region: Region.US_EAST, date: "2026-01-29" },
  { serviceName: "BE Service EU", status: Status.DEGRADED, responseTime: 420, region: Region.EU_WEST, date: "2026-04-16" },
  { serviceName: "FE Service EU", status: Status.DEGRADED, responseTime: 380, region: Region.EU_WEST, date: "2026-04-14" }
]
```

Tomando en cuenta estos resultados, se quiere realizar lo siguiente:

- Obtener un arreglo con todas las regiones donde se tengan servidores con un estatus UP.
- Obtener un objeto cuyos atributos serán los nombres de los servicios que estén en DEGRADED. El contenido de esos atributos será la cantidad de días que lleva ese servicio en DEGRADED.
- Obtener un arreglo que contiene en cada una de sus posiciones el promedio en ms del tiempo de respuesta de los servidores que se encuentren en UP y DEGRADED según su región. La posición donde debe ingresarse el promedio debe ser elegida tomando en cuenta el enum Region
- El promedio en ms del tiempo de respuesta de los servidores que se encuentran en UP
- Se puede notar que el BE Service y FE Service tienen versiones desplegadas en EU_WEST que están en DEGRADED. Esto se debe a que se quiere centralizar toda la operación de la empresa en un solo servidor sin tener distinción entre regiones. Para esto se quiere eliminar todos los servidores DEGRADED en EU_WEST, actualizar el BE Service y el FE Service para que ahora estén en EU_WEST, actualizando la fecha que tiene a la actual.